

Algorithm Design

COP 3502C – Computer Science I

Spring 2026

Yancy Vance Paredes, PhD

Problem Statement

Given a sorted list of **N** numbers, check if **X** is in the list

10, 20, 30, 40, 50, 60, 70, 80

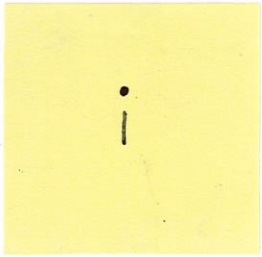
Is there an **80** in the list?

.

Strategy 1 – Linear Search

Is there an 80 in the list?

10, 20, 30, 40, 50, 60, 70, 80



10	20	30	40	50	60	70	80
0	1	2	3	4	5	6	7

Strategy 2 – Binary Search

Is there an 80 in the list?

10, 20, 30, 40, 50, 60, 70, 80

left

right

10	20	30	40	50	60	70	80
0	1	2	3	4	5	6	7

mid

Scenarios /1

Worst Case

The maximum amount of work for a given input size

Best Case

The minimum amount of work for a given input size

Scenarios /2

Average Case

The expected amount of work over all possible inputs

Discussion

In algorithm analysis, we often focus on the **worst-case scenario**

Your Turn! /1

Provide example **query** values that result in the best-case and the worst-case scenarios for **linear search** and **binary search**.

Your Turn! /2

Implement a **recursive** version of the binary search algorithm.

Problem Statement

Given **two sorted lists**, how many values are **common** to both?

For example:

10, 20, 30, 40

5, 10, 15, 20

The output is:

2

Discussion

- Assume that the two lists have equal length and within a list, the elements are distinct
- What is your strategy?
- What is the running time of your solution?

Strategy 1

10, 20, 30, 40

n

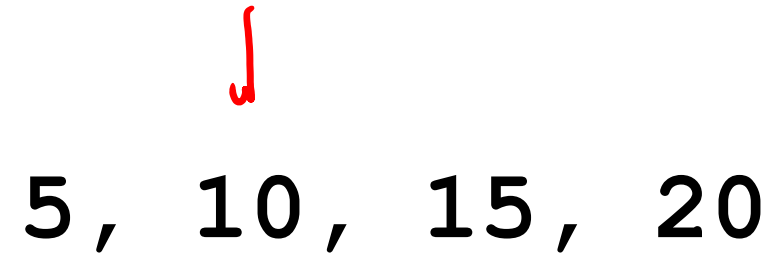
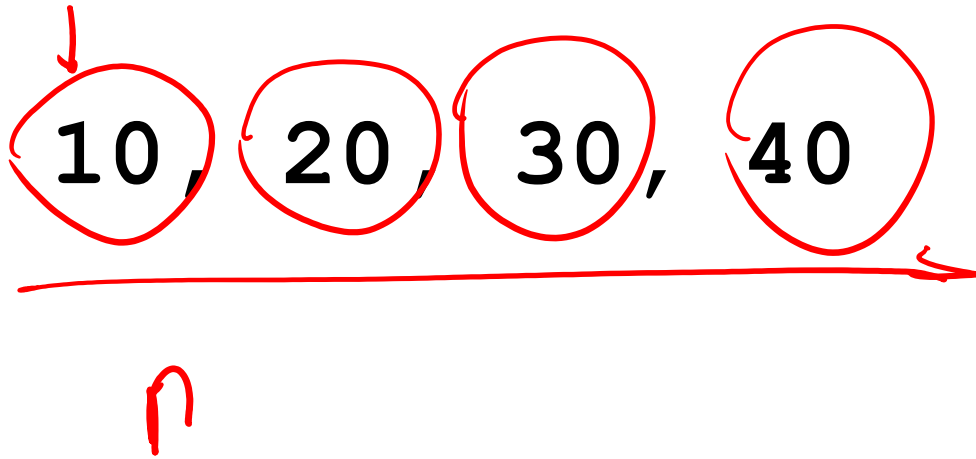
$$n \cdot n = (n^2)$$

5, 10, 15, 20

$O(n)$
each

$$T(n) = O(n^2)$$

Strategy 2



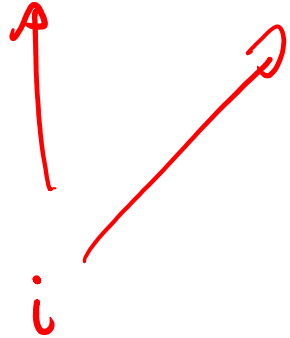
$O(\lg n)$
each

$$T(n) = O(n \lg n)$$

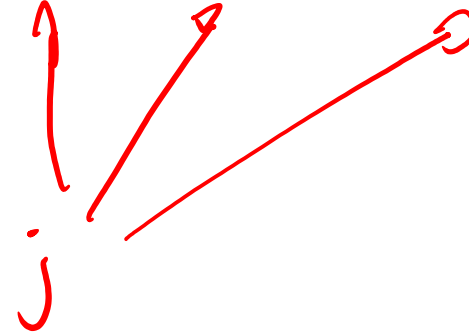
Strategy 3

$$T(n) = O(n)$$

10, 20, 30, 40



5, 10, 15, 20



check if true, if yes, continue
if not advance index of lower value

10 20 30 40

15 25 35 45

i

10	20	30	40
0	1	2	3

j

5	10	15	20
0	1	2	3

Discussion /1

- Imagine you have two **disjoint** lists where the size is **1,000,000**
- Which of the 3 strategies will finish first?

Discussion /2

m
 $arr1$

n
 $arr2$

What if the two lists don't have the same lengths? How will this affect the running time of your solution?

if $m = n$

if $m \neq n$

- Strategy 1

$$O(n^2)$$

$$O(mn)$$

- Strategy 2

$$O(n \lg n)$$

$$O(m \lg n) \text{ or } O(n \lg m)$$

- ~~✗~~ • Strategy 3

$$O(n)$$

$$O(m+n)$$

Your Turn!

Implement the three strategies we analyzed. The $O(N)$ strategy is an extension of the linear search. The $O(N \lg N)$ strategy is an extension of the binary search.

Challenge

How would the code change if we were working with sorted arrays of strings instead?

Final Notes /1

- The example we saw may not necessarily be the worst-case scenario for a strategy
- For each strategy, think what the worst-case scenario looks like
- The running time we identified are for the worst-case scenarios

Final Notes /2

- The problem we explored is often referred to as **sorted array matching problem**
- It can also be referred to as **intersection** of two sorted arrays
- The problem may come in different forms, but the idea is the same

Questions?